Case Study on PAYXPERT

PRASANTH.A

1.Introduction

A Payroll management system is a digital solution that streamlines the management of payroll in an organization. It simplifies the entire payroll process– from managing employee information, salary calculations, tax calculations, and financial information.

The system is built using strong principles of Object-Oriented programming (OOP) and database design (SQL), following modular coding practices for maintainability and scalability purpose. This project touches key concepts such as control flow, loops, collections, exception handling and unit testing, so lends itself as a good educational case study for would-be software developers.

PayXpert offers the following core functionalities:

Management of employee information

Processing Salaries and Deductions Automatically

Income bracket based Tax calculation

Recordkeeping and Financial Reporting

The app is built using Python And MYSQL

2. Purpose of the Project

The primary purpose of the **PayXpert Payroll Management System** is to develop an efficient, reliable, and scalable payroll processing solution that simplifies the complex tasks involved in employee compensation management. The project aims to address the challenges faced by HR and finance departments in handling payroll operations manually or with outdated systems.

This project serves both academic and practical purposes:

Project Goals

- To automate the complete payroll lifecycle—from employee onboarding to salary disbursement.
- To ensure **accuracy and transparency** in salary, tax, and deduction calculations.
- To provide real-time access to **employee financial records** and payroll history.
- To support easy generation of **financial and tax reports** for compliance and analysis.

Learning Objectives

- Apply **Python programming** and **MySQL database** concepts to build a real-world application.
- Understand and implement **Object-Oriented Programming (OOP)** in a modular project.
- Use **SQL** for designing relational schemas and performing database operations.
- Implement **collections**, **exception handling**, **validation**, and **unit testing** in Python.
- Develop a **menu-driven interface** to interact with the system.

By the end of the project, the system will demonstrate how Python and SQL can be used together to solve real-world business problems effectively while reinforcing strong software engineering principles.

3. Scope of the Project

The **PayXpert Payroll Management System** covers a wide range of features and technical components, designed to manage payroll processes efficiently using Python and MySQL. This project not only automates payroll tasks but also demonstrates how to build a well-structured, maintainable application using real-world development principles.

Here's what's included in the scope of this project:

Entity Modeling

All the main data objects in the system—like employees, payroll records, tax information, and financial transactions—are represented using Python classes. These classes are placed in the entity package and include:

- Private attributes for each field (like name, date of joining, salary, etc.)
- Constructors to create objects easily
- Getter and setter methods to access or update values
- A few utility methods like calculateAge() in the Employee class

This makes the system easy to manage and extend in the future.

Data Access Layer (DAO)

The Data Access Layer is where the actual database work happens. It includes interfaces and service classes for:

- Adding, updating, retrieving, and deleting records
- Handling different tables like Employee, Payroll, Tax, and FinancialRecord

Each major module (like employee or payroll) has its own interface (like IEmployeeService) and a class that implements it (like EmployeeServiceImpl). This keeps the database logic separate from the

rest of the code, which makes everything more modular and easier to maintain.

Custom Exceptions

To make the program more reliable and user-friendly, custom exceptions are used to handle specific errors. These are stored in the exception package and include:

- EmployeeNotFoundException for missing employee data
- PayrollGenerationException for payroll issues
- TaxCalculationException for tax errors
- FinancialRecordException for financial data problems
- InvalidInputException for bad input
- DatabaseConnectionException for connection failures

These help make the program more robust and improve error tracking.

Database Integration

The application connects to a MySQL database to store all employee and payroll information. This is managed through two key utility classes:

- DBPropertyUtil: Reads the database connection settings from a property file
- DBConnUtil: Uses that information to create a connection to the database

All the tables—like Employee, Payroll, Tax, and FinancialRecord—are properly structured with primary and foreign keys to keep the data organized and consistent.

Functionalities

The system includes all the key features you'd expect in a payroll system:

- Managing employee records (add, update, delete, view)
- Generating payrolls with salary, overtime, and deductions
- Calculating taxes based on income
- Recording financial transactions like expenses or tax payments
- Generating reports for salaries, taxes, and financial summaries

Each of these features is available through simple, reusable functions that follow best coding practices.

Menu-Driven Application

The application is interactive and user-friendly. It includes a menu-driven interface in the main package where users can choose what they want to do:

- Add a new employee
- Generate payroll
- View financial records
- And much more

Everything is validated and all exceptions are handled neatly to make sure the program doesn't crash on bad input.

Unit Testing

To make sure the system works correctly, unit tests are written using Python's built-in unittest module. These tests check:

- Whether the salary and tax calculations are correct
- How the system behaves with invalid inputs
- Whether different modules are working as expected

Testing helps ensure the system is reliable and ready for real use.

4. Structure of the Project

The PayXpert Payroll Management System is organized in a way that keeps everything modular, maintainable, and easy to navigate. It's split into different packages and layers, each handling specific responsibilities like entity modeling, database access, exception handling, utility functions, and the main application flow.

This section focuses on the **SQL structure** of the project, which forms the backbone for storing and managing all the data.

4.1 SQL Structure (Database Schema)

The system uses a **MySQL relational database** to store employee data, payroll information, taxes, and financial records. The schema is designed using normalization principles to avoid data redundancy and ensure consistency through primary and foreign keys.

DATABASE AND TABLE CREATION

```
create database payxpert1;
use payxpert1;

create table employee (
Employee_ID int not null auto_increment,
First_Name varchar(20),
Last_Name varchar(20),
Date_of_Birth date,
Gender varchar(10),
Email varchar(25),
Phone_Number varchar(15),
Address varchar(50),
Position varchar(20),
Joining_Date date,
Termination_Date date,
primary key(Employee_ID)
);

create table payroll(
```

```
Payroll_ID int not null auto_increment,
Employee_ID int not null,
Payperiod_Start_Date date,
Payperiod_End_Date date ,
Basic_Salary decimal(10,2),
Overtime_Pay decimal(10,2),
Deductions decimal(10,2),
Net_Salary decimal(10,2),
primary key(Payroll_ID),
foreign key(Employee_ID) references employee(Employee_ID) on delete
cascade
);

create table tax(
Tax_ID int not null auto_increment,
Employee_ID int not null,
Tax_Year int,
Taxable_Income decimal(10,2),
Tax_Amount decimal(10,2),
primary key(Tax_ID),
foreign key(Employee_ID) references employee(Employee_ID) on delete
cascade
);

create table financialrecord(
Record_ID int not null auto_increment,
Employee_ID int not null,
Record_Date date,
Description_Category varchar(30),
Amount decimal(10,2),
Record_Type varchar(20),
primary key(Record_ID),
foreign key(Employee_ID) references employee(Employee_ID) on delete
cascade
);

insert into employee
(First_Name,Last_Name,Date_of_Birth,Gender,Email,Phone_Number,
Address,Position, Joining_Date, Termination_Date) values
('John','Doe','1985-02-15','Male','john.doe@email.com','9876543210','New
York','Manager','2020-01-10',NULL),
('Jane','Smith','1990-06-
20','Female','jane.smith@email.com','9123456789','Chicago','Developer','2
021-03-22',NULL),
('Mike','Brown','1988-12-
12','Male','mike.brown@email.com','9988776655','Houston','Tester','2022-
07-01',NULL),
```

('Sara','Lee','1992-08-
08','Female','sara.lee@email.com','9871234560','Dallas','Analyst','2023-01-
15',NULL),
('Tom','Wilson','1984-10-
10','Male','tom.wilson@email.com','9090909090','Seattle','Designer','2019-
05-25',NULL),
('Emily','Clark','1995-11-
30','Female','emily.clark@email.com','9988771122','San Jose','HR','2022-
09-05',NULL),
('Robert','King','1980-04-
04','Male','robert.king@email.com','9876000000','Atlanta','Admin','2018-
12-10','2023-12-31'),
('Lucy','Turner','1993-03-
03','Female','lucy.turner@email.com','9012345678','Boston','Support','2020
-06-18',NULL),
('Steve','Jobs','1980-02-
24','Male','steve.jobs@email.com','9091234567','California','CEO','2010-01-
01',NULL),
('Nina','Patel','1997-07-
07','Female','nina.patel@email.com','9911223344','Phoenix','Intern','2024-
02-01',NULL);

insert into payroll
(Employee_ID,Payperiod_Start_Date,Payperiod_End_Date,
Basic_Salary, Overtime_Pay,Deductions, Net_Salary) values
(1,'2024-03-01','2024-03-31',70000.00,5000.00,2000.00,73000.00),
(2,'2024-03-01','2024-03-31',60000.00,3000.00,1500.00,61500.00),
(3,'2024-03-01','2024-03-31',50000.00,2000.00,1000.00,51000.00),
(4,'2024-03-01','2024-03-31',55000.00,2500.00,1800.00,55700.00),
(5,'2024-03-01','2024-03-31',65000.00,4000.00,2200.00,66800.00),
(6,'2024-03-01','2024-03-31',48000.00,1200.00,1000.00,48200.00),
(7,'2024-03-01','2024-03-31',40000.00,0.00,500.00, 39500.00),
(8,'2024-03-01','2024-03-31',43000.00,1000.00,700.00,43300.00),
(9,'2024-03-01','2024-03-31',90000.00,10000.00,3000.00,97000.00),
(10,'2024-03-01','2024-03-31',20000.00,500.00,200.00,20300.00);

insert into tax (Employee_ID,Tax_Year,Taxable_Income,Tax_Amount)
values
(1,2023,840000.00,84000.00),
(2,2023,720000.00,72000.00),
(3,2023,600000.00,60000.00),
(4,2023,660000.00,66000.00),
(5,2023,780000.00,78000.00),
(6,2023,576000.00,57600.00),
(7,2023,480000.00,48000.00),
(8,2023,516000.00,51600.00),
(9,2023,1080000.00,108000.00),

```
(10,2023,240000.00,24000.00);

insert into financialrecord
(Employee_ID,Record_Date,Description_Category,Amount,Record_Type)
values
(1,'2024-03-05','Travel',1500.00,'Expense'),
(2,'2024-03-10','Bonus',3000.00,'Income'),
(3,'2024-03-12','Lunch',500.00,'Expense'),
(4,'2024-03-15','Project Allowance',2000.00,'Income'),
(5,'2024-03-18','Conference',800.00,'Expense'),
(6,'2024-03-20','Referral Bonus',1000.00,'Income'),
(7,'2024-03-22','Training',1200.00,'Expense'),
(8,'2024-03-24','Performance Bonus',2500.00,'Income'),
(9,'2024-03-26','Medical',900.00,'Expense'),
(10,'2024-03-28','Stipend',1500.00,'Income');
```

4.2 OOP Structure (Object-Oriented Programming)

The **PayXpert Payroll Management System** is designed using core **Object-Oriented Programming (OOP)** principles in Python. This structure allows the system to be modular, reusable, maintainable, and easy to extend. The entire application is organized into packages based on responsibility, and each class represents a real-world entity or logical functionality.

Below is an overview of how OOP is applied in this project:

1. Entity Module (entity):

The entity module contains all the data model classes that represent real-world objects in the system, such as Employee, Payroll, Tax, and FinancialRecord.
These classes define the structure of data with private attributes, constructors, getters, and setters. They do not contain any business logic and are used to transfer data between different layers of the application.

Employee.py

```
class Employee:
    def __init__(self, employee_id=None, first_name=None,
last_name=None, date_of_birth=None,
                 gender=None, email=None, phone_number=None,
address=None,
                 position=None, joining_date=None, termination_date=None):
        self.__employee_id = employee_id
        self.__first_name = first_name
        self.__last_name = last_name
        self.__date_of_birth = date_of_birth
        self.__gender = gender
        self.__email = email
        self.__phone_number = phone_number
        self.__address = address
        self.__position = position
        self.__joining_date = joining_date
        self.__termination_date = termination_date

    # Getters and setters
    def get_employee_id(self):
        return self.__employee_id
    def set_employee_id(self, value):
        self.__employee_id = value

    def get_first_name(self):
        return self.__first_name
    def set_first_name(self, value):
        self.__first_name = value

    def get_last_name(self):
```

```
    return self.__last_name
def set_last_name(self, value):
    self.__last_name = value

def get_date_of_birth(self):
    return self.__date_of_birth
def set_date_of_birth(self, value):
    self.__date_of_birth = value

def get_gender(self):
    return self.__gender
def set_gender(self, value):
    self.__gender = value

def get_email(self):
    return self.__email
def set_email(self, value):
    self.__email = value

def get_phone_number(self):
    return self.__phone_number
def set_phone_number(self, value):
    self.__phone_number = value

def get_address(self):
    return self.__address
def set_address(self, value):
    self.__address = value

def get_position(self):
    return self.__position
def set_position(self, value):
    self.__position = value

def get_joining_date(self):
    return self.__joining_date
def set_joining_date(self, value):
    self.__joining_date = value

def get_termination_date(self):
    return self.__termination_date
def set_termination_date(self, value):
    self.__termination_date = value
```

```
    # Method
    def calculate_age(self):
        if self.__date_of_birth:
            today = date.today()
            return today.year - self.__date_of_birth.year - (
                (today.month, today.day) < (self.__date_of_birth.month,
self.__date_of_birth.day))
        return None
```

FinancialRecord.py

```
class FinancialRecord:
    def __init__(self, record_id=None, employee_id=None,
record_date=None,
                 description=None, amount=0.0, record_type=None):
        self.__record_id = record_id
        self.__employee_id = employee_id
        self.__record_date = record_date
        self.__description = description
        self.__amount = amount
        self.__record_type = record_type

    def get_record_id(self):
        return self.__record_id
    def set_record_id(self, value):
        self.__record_id = value

    def get_employee_id(self):
        return self.__employee_id
    def set_employee_id(self, value):
        self.__employee_id = value

    def get_record_date(self):
        return self.__record_date
    def set_record_date(self, value):
        self.__record_date = value

    def get_description(self):
        return self.__description
    def set_description(self, value):
        self.__description = value
```

```
    def get_amount(self):
        return self.__amount
    def set_amount(self, value):
        self.__amount = value

    def get_record_type(self):
        return self.__record_type
    def set_record_type(self, value):
        self.__record_type = value
```

Payroll.py

```
class Payroll:
    def __init__(self, payroll_id=None, employee_id=None,
pay_period_start_date=None,
                 pay_period_end_date=None, basic_salary=0.0,
overtime_pay=0.0,
                 deductions=0.0, net_salary=0.0):
        self.__payroll_id = payroll_id
        self.__employee_id = employee_id
        self.__pay_period_start_date = pay_period_start_date
        self.__pay_period_end_date = pay_period_end_date
        self.__basic_salary = basic_salary
        self.__overtime_pay = overtime_pay
        self.__deductions = deductions
        self.__net_salary = net_salary

    def get_payroll_id(self):
        return self.__payroll_id
    def set_payroll_id(self, value):
        self.__payroll_id = value

    def get_employee_id(self):
        return self.__employee_id
    def set_employee_id(self, value):
        self.__employee_id = value

    def get_pay_period_start_date(self):
        return self.__pay_period_start_date
    def set_pay_period_start_date(self, value):
```

```
        self.__pay_period_start_date = value

    def get_pay_period_end_date(self):
        return self.__pay_period_end_date
    def set_pay_period_end_date(self, value):
        self.__pay_period_end_date = value

    def get_basic_salary(self):
        return self.__basic_salary
    def set_basic_salary(self, value):
        self.__basic_salary = value

    def get_overtime_pay(self):
        return self.__overtime_pay
    def set_overtime_pay(self, value):
        self.__overtime_pay = value

    def get_deductions(self):
        return self.__deductions
    def set_deductions(self, value):
        self.__deductions = value

    def get_net_salary(self):
        return self.__net_salary
    def set_net_salary(self, value):
        self.__net_salary = value
```

Tax.py

```
class Tax:
    def __init__(self, tax_id=None, employee_id=None, tax_year=None,
                 taxable_income=0.0, tax_amount=0.0):
        self.__tax_id = tax_id
        self.__employee_id = employee_id
        self.__tax_year = tax_year
        self.__taxable_income = taxable_income
        self.__tax_amount = tax_amount

    def get_tax_id(self):
        return self.__tax_id
```

```
    def set_tax_id(self, value):
        self.__tax_id = value

    def get_employee_id(self):
        return self.__employee_id
    def set_employee_id(self, value):
        self.__employee_id = value

    def get_tax_year(self):
        return self.__tax_year
    def set_tax_year(self, value):
        self.__tax_year = value

    def get_taxable_income(self):
        return self.__taxable_income
    def set_taxable_income(self, value):
        self.__taxable_income = value

    def get_tax_amount(self):
        return self.__tax_amount
    def set_tax_amount(self, value):
        self.__tax_amount = value
```

2. DAO Module (dao/):

The dao module (Data Access Object) contains all the interfaces and service classes responsible for interacting with the database.
It includes method definitions for operations like adding, updating, deleting, and retrieving data.

This layer separates the database logic from the rest of the application, making the code more modular and easier to manage.

employee_service.py

```
from dao.iemployee_service import IEmployeeService
from entity.Employee import Employee

class EmployeeService(IEmployeeService):
    def __init__(self, connection):
        self.conn = connection

    def get_employee_by_id(self, employee_id):
        cursor = self.conn.cursor()
        cursor.execute("SELECT * FROM employee WHERE Employee_ID =
%s", (employee_id,))
        row = cursor.fetchone()
        if row:
            return Employee(
                employee_id=row['Employee_ID'],
                first_name=row['First_Name'],
                last_name=row['Last_Name'],
                date_of_birth=row['Date_of_Birth'],
                gender=row['Gender'],
                email=row['Email'],
                phone_number=row['Phone_Number'],
                address=row['Address'],
                position=row['Position'],
                joining_date=row['Joining_Date'],
                termination_date=row['Termination_Date']
            )

        return None

    def get_all_employees(self):
        cursor = self.conn.cursor()
        cursor.execute("SELECT * FROM employee")
        rows = cursor.fetchall()
        return [Employee(
            employee_id=row['Employee_ID'],
            first_name=row['First_Name'],
            last_name=row['Last_Name'],
            date_of_birth=row['Date_of_Birth'],
            gender=row['Gender'],
            email=row['Email'],
            phone_number=row['Phone_Number'],
```

```
            address=row['Address'],
            position=row['Position'],
            joining_date=row['Joining_Date'],
            termination_date=row['Termination_Date']
        ) for row in rows]

    def add_employee(self, employee):
        cursor = self.conn.cursor()
        query = """
            INSERT INTO employee (Employee_ID, First_Name, Last_Name,
Date_of_Birth, Gender,
                                  Email, Phone_Number, Address, Position,
Joining_Date, Termination_Date)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
        """
        values = (
            employee.get_employee_id(), employee.get_first_name(),
employee.get_last_name(),
            employee.get_date_of_birth(), employee.get_gender(),
employee.get_email(),
            employee.get_phone_number(), employee.get_address(),
employee.get_position(),
           employee.get_joining_date(), employee.get_termination_date()
        )
        cursor.execute(query, values)
        self.conn.commit()

    def update_employee(self, employee):
        cursor = self.conn.cursor()
        query = """
            UPDATE employee
           SET First_Name=%s, Last_Name=%s, Email=%s, Phone_Number=%s
            WHERE Employee_ID = %s
        """
        values = (
            employee.get_first_name(), employee.get_last_name(),
            employee.get_email(), employee.get_phone_number(),
            employee.get_employee_id()
        )
        cursor.execute(query, values)
        self.conn.commit()

    def remove_employee(self, employee_id):
        cursor = self.conn.cursor()
```

```
        cursor.execute("DELETE FROM employee WHERE Employee_ID = %s",
(employee_id,))
        self.conn.commit()
```

financialrecord_service.py

```
from dao.ifinancialrecord_service import IFinancialRecordService

class FinancialRecordService(IFinancialRecordService):
    def __init__(self, connection):
        self.conn = connection

    def add_financial_record(self, record):
        self.records[record.record_id] = record

    def get_financial_record_by_id(self, record_id):
        return self.records.get(record_id)

    def get_financial_records_for_employee(self, employee_id):
        return [r for r in self.records.values() if r.employee_id ==
employee_id]

    def get_financial_records_for_date(self, date):
        return [r for r in self.records.values() if r.record_date == date]
```

iemployee_service.py

```
from abc import ABC, abstractmethod

class IEmployeeService(ABC):
    @abstractmethod
    def get_employee_by_id(self, employee_id):
        pass

    @abstractmethod
    def get_all_employees(self):
        pass
```

```
    @abstractmethod
    def add_employee(self, employee):
        pass

    @abstractmethod
    def update_employee(self, employee):
        pass

    @abstractmethod
    def remove_employee(self, employee_id):
        pass
```

ifinancialrecord_service.py

```
from abc import ABC, abstractmethod

class IFinancialRecordService(ABC):
    @abstractmethod
    def add_financial_record(self, record):
        pass

    @abstractmethod
    def get_financial_record_by_id(self, record_id):
        pass

    @abstractmethod
    def get_financial_records_for_employee(self, employee_id):
        pass

    @abstractmethod
    def get_financial_records_for_date(self, date):
        pass
```

ipayroll_service.py

```
from abc import ABC, abstractmethod

class IPayrollService(ABC):
    @abstractmethod
    def generate_payroll(self, employee):
        pass
```

```
    @abstractmethod
    def get_payroll_by_id(self, payroll_id):
        pass

    @abstractmethod
    def get_payrolls_for_employee(self, employee_id):
        pass

    @abstractmethod
    def get_payrolls_for_period(self, start_date, end_date):
        pass
```

itax_service.py

```
from abc import ABC, abstractmethod

class ITaxService(ABC):
    @abstractmethod
    def calculate_tax(self, employee):
        pass

    @abstractmethod
    def get_tax_by_id(self, tax_id):
        pass

    @abstractmethod
    def get_taxes_for_employee(self, employee_id):
        pass

    @abstractmethod
    def get_taxes_for_year(self, year):
        pass
```

payroll_service.py

```
from dao.ipayroll_service import IPayrollService
from entity.Payroll import Payroll

class PayrollService(IPayrollService):
    def __init__(self, connection):
        self.conn = connection
```

```
    def generate_payroll(self, payroll: Payroll):
        cursor = self.conn.cursor()
        query = """
            INSERT INTO payroll (Employee_ID, Payperiod_Start_Date,
Payperiod_End_Date,
                                 Basic_Salary, Overtime_Pay, Deductions,
Net_Salary)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
        """
        values = (
            payroll.get_employee_id(),
            payroll.get_pay_period_start_date(),
            payroll.get_pay_period_end_date(),
            payroll.get_basic_salary(),
            payroll.get_overtime_pay(),
            payroll.get_deductions(),
            payroll.get_net_salary()
        )
        cursor.execute(query, values)
        self.conn.commit()

    def get_payroll_by_id(self, payroll_id):
        return self.payrolls.get(payroll_id)

    def get_payrolls_for_employee(self, employee_id):
        cursor = self.conn.cursor()
        cursor.execute("SELECT * FROM payroll WHERE Employee_ID = %s",
(employee_id,))
        rows = cursor.fetchall()

        from entity.Payroll import Payroll
        return [Payroll(
            payroll_id=row["Payroll_ID"],
            employee_id=row["Employee_ID"],
            pay_period_start_date=row["Payperiod_Start_Date"],
            pay_period_end_date=row["Payperiod_End_Date"],
            basic_salary=row["Basic_Salary"],
            overtime_pay=row["Overtime_Pay"],
            deductions=row["Deductions"],
            net_salary=row["Net_Salary"]
        ) for row in rows]

    def get_payrolls_for_period(self, start_date, end_date):
```

```
        return [p for p in self.payrolls.values() if start_date <=
p.pay_period_start_date <= end_date]
```

tax_service.py

```
from dao.itax_service import ITaxService
from entity.Tax import Tax

class TaxService(ITaxService):
    def __init__(self, connection):
        self.conn = connection

    def calculate_tax(self, employee_id, tax_year):
        from decimal import Decimal
        cursor = self.conn.cursor()

        # Calculate taxable income from payroll
        cursor.execute("""
            SELECT SUM(Basic_Salary + Overtime_Pay - Deductions) AS
TaxableIncome
            FROM payroll
            WHERE Employee_ID = %s
        """, (employee_id,))
        result = cursor.fetchone()

        taxable_income = result["TaxableIncome"] if
result["TaxableIncome"] else Decimal('0.00')
        tax_amount = taxable_income * Decimal('0.10')  # 10% tax

        # Insert into tax table
        cursor.execute("""
            INSERT INTO tax (Employee_ID, Tax_Year, Taxable_Income,
Tax_Amount)
            VALUES (%s, %s, %s, %s)
        """, (employee_id, tax_year, taxable_income, tax_amount))

        self.conn.commit()

    def get_taxes_for_employee(self, employee_id):
        cursor = self.conn.cursor()
        cursor.execute("SELECT * FROM tax WHERE Employee_ID = %s",
(employee_id,))
```

```
        rows = cursor.fetchall()

        return [
            Tax(
                tax_id=row["Tax_ID"],
                employee_id=row["Employee_ID"],
                tax_year=row["Tax_Year"],
                taxable_income=row["Taxable_Income"],
                tax_amount=row["Tax_Amount"]
            )
            for row in rows
        ]

    def get_tax_by_id(self, tax_id):
        cursor = self.conn.cursor()
        cursor.execute("SELECT * FROM tax WHERE Tax_ID = %s", (tax_id,))
        row = cursor.fetchone()

        if row:
            return Tax(
                tax_id=row["Tax_ID"],
                employee_id=row["Employee_ID"],
                tax_year=row["Tax_Year"],
                taxable_income=row["Taxable_Income"],
                tax_amount=row["Tax_Amount"]
            )
        return None

    def get_taxes_for_year(self, tax_year):
        cursor = self.conn.cursor()
        cursor.execute("SELECT * FROM tax WHERE Tax_Year = %s",
(tax_year,))
        rows = cursor.fetchall()

        return [
            Tax(
                tax_id=row["Tax_ID"],
                employee_id=row["Employee_ID"],
                tax_year=row["Tax_Year"],
                taxable_income=row["Taxable_Income"],
                tax_amount=row["Tax_Amount"]
            )
            for row in rows
        ]
```

3.Exception Module (exception/):

The exception module contains all the custom exception classes used in the application.

These exceptions handle specific error scenarios such as missing employee records, failed payroll generation, invalid input, or database connection issues.

Using custom exceptions helps in managing errors more clearly and improves the reliability of the system.

custom_exceptions.py

```
class EmployeeNotFoundException(Exception):
    def __init__(self, message="Employee not found."):
        super().__init__(message)

class PayrollGenerationException(Exception):
    def __init__(self, message="Error generating payroll."):
        super().__init__(message)

class TaxCalculationException(Exception):
    def __init__(self, message="Error calculating tax."):
        super().__init__(message)

class FinancialRecordException(Exception):
    def __init__(self, message="Error in financial record operation."):
        super().__init__(message)

class InvalidInputException(Exception):
    def __init__(self, message="Invalid input provided."):
        super().__init__(message)

class DatabaseConnectionException(Exception):
    def __init__(self, message="Failed to connect to the database."):
        super().__init__(message)
```

5. Utility Module (util/):

The util module contains helper classes that handle common utility functions, such as managing database connections and reading configuration files.

It typically includes:

- DBPropertyUtil for reading database settings from a property file.

- DBConnUtil for creating and returning a live database connection.

These utility classes keep the code clean and make it easier to manage configuration and connectivity across the application.

database_context.py

```
import pymysql

class DatabaseContext:
    @staticmethod
    def get_connection():
        try:
            connection = pymysql.connect(
                host='localhost',
                user='root',
                password='root',  # replace with your actual password
                database='payxpert1',      # your database name
                cursorclass=pymysql.cursors.DictCursor
            )
            return connection
        except pymysql.MySQLError as e:
            print("Database connection failed:", e)
            return None
```

report_generator.py

```
class ReportGenerator:
    @staticmethod
    def generate_payroll_report(payrolls):
        for p in payrolls:
            print(f"{p.payroll_id} - {p.employee_id} - {p.net_salary}")

    @staticmethod
    def generate_tax_summary(taxes):
        for t in taxes:
            print(f"{t.tax_id} - {t.employee_id} - {t.tax_amount}")

    @staticmethod
    def generate_financial_report(records):
        for r in records:
            print(f"{r.record_id} - {r.description} - {r.amount}
({r.record_type})")
```

validation_service.py

```
import re

class ValidationService:
    @staticmethod
    def validate_email(email):
        return bool(re.match(r"[^@]+@[^@]+\.[^@]+", email))

    @staticmethod
    def validate_phone(phone):
        return phone.isdigit() and len(phone) == 10
```

5.Main Application (app/):

The app module contains the entry point of the application.
It includes a menu-driven interface that allows users to interact with the system and perform various operations like adding employees, generating payroll, and calculating taxes.

This module connects all other modules and demonstrates the complete functionality of the Payroll Management System.

MainModule.py

```
import sys
import os
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__)
, '..')))
from dao.employee_service import EmployeeService
from dao.payroll_service import PayrollService
from dao.tax_service import TaxService
from dao.financialrecord_service import FinancialRecordService
from util.database_context import DatabaseContext
from exception.custom_exceptions import *
from entity.Employee import Employee
from entity.Payroll import Payroll
from entity.Tax import Tax

import datetime

class MainModule:
    def __init__(self):
        # Establishing DB connection
        self.connection = DatabaseContext.get_connection()
        if not self.connection:
            raise DatabaseConnectionException("Could not connect to
database.")

        # Injecting DB connection into services
        self.employee_service = EmployeeService(self.connection)
        self.payroll_service = PayrollService(self.connection)
        self.tax_service = TaxService(self.connection)
        self.financial_service = FinancialRecordService(self.connection)

    def menu(self):
        while True:
            print("\n========== EMPLOYEE MANAGEMENT SYSTEM
==========")
            print("1. Add Employee")
            print("2. View All Employees")
            print("3. Get Employee by ID")
```

```
        print("4. Update Employee")
        print("5. Delete Employee")
        print("6. Generate Payroll")
        print("7. View Payrolls for an Employee")
        print("8. Calculate Tax")
        print("9. View Taxes for an Employee")
        print("10. Exit")

        choice = input("Enter your choice: ")

        try:
            if choice == '1':
                self.add_employee()
            elif choice == '2':
                self.view_all_employees()
            elif choice == '3':
                self.get_employee_by_id()
            elif choice == '4':
                self.update_employee()
            elif choice == '5':
                self.delete_employee()
            elif choice == '6':
                self.generate_payroll()
            elif choice == '7':
                self.view_payrolls_for_employee()
            elif choice == '8':
                self.calculate_tax()
            elif choice == '9':
                self.view_taxes_for_employee()
            elif choice == '10':
                print("Exiting...")
                break


            else:
                print("Invalid choice. Try again.")

        except Exception as e:
            print(f"Error: {str(e)}")

    def add_employee(self):
        print("\nEnter Employee Details:")
        try:
            emp = Employee(
```

```
            employee_id=input("Employee ID: "),
            first_name=input("First Name: "),
            last_name=input("Last Name: "),
            date_of_birth=input("Date of Birth (YYYY-MM-DD): "),
            gender=input("Gender: "),
            email=input("Email: "),
            phone_number=input("Phone Number: "),
            address=input("Address: "),
            position=input("Position: "),
            joining_date=input("Joining Date (YYYY-MM-DD): "),
            termination_date=input("Termination Date (YYYY-MM-DD) or
leave blank: ") or None
        )
        self.employee_service.add_employee(emp)
        print("Employee added successfully.")
    except InvalidInputException as e:
        print(f"Invalid input: {str(e)}")

def view_all_employees(self):
    employees = self.employee_service.get_all_employees()
    if not employees:
        print("No employees found.")
    for emp in employees:
        print(f"ID: {emp.get_employee_id()} | Name:
{emp.get_first_name()} {emp.get_last_name()} | Email:
{emp.get_email()}")

def get_employee_by_id(self):
    emp_id = input("Enter Employee ID: ")
    emp = self.employee_service.get_employee_by_id(emp_id)
    if emp:
        print(f"""
        Employee Details:
        ID: {emp.get_employee_id()}
        First Name: {emp.get_first_name()}
        Last Name: {emp.get_last_name()}
        Date of Birth: {emp.get_date_of_birth()}
        Gender: {emp.get_gender()}
        Email: {emp.get_email()}
        Phone Number: {emp.get_phone_number()}
        Address: {emp.get_address()}
        Position: {emp.get_position()}
        Joining Date: {emp.get_joining_date()}
        Termination Date: {emp.get_termination_date()}
```

```
        """)
    else:
        print("Employee not found.")

    def update_employee(self):
        emp_id = input("Enter Employee ID to update: ")
        employee = self.employee_service.get_employee_by_id(emp_id)
        if not employee:
            print("Employee not found.")
            return
        print("Leave field blank to keep current value.")
        employee.set_first_name(input("First Name: ") or
employee.get_first_name())
        employee.set_last_name(input("Last Name: ") or
employee.get_last_name())
        employee.set_email(input("Email: ") or employee.get_email())
        employee.set_phone_number(input("Phone Number: ") or
employee.get_phone_number())
        self.employee_service.update_employee(employee)
        print("Employee updated.")

    def delete_employee(self):
        emp_id = input("Enter Employee ID to delete: ")
        self.employee_service.remove_employee(emp_id)
        print("Employee deleted.")

    def generate_payroll(self):
        emp_id = input("Enter Employee ID: ")
        start_date = input("Enter Pay Period Start Date (YYYY-MM-DD): ")
        end_date = input("Enter Pay Period End Date (YYYY-MM-DD): ")
        basic_salary = float(input("Enter Basic Salary: "))
        overtime = float(input("Enter Overtime Pay: "))
        deductions = float(input("Enter Deductions: "))

        # Calculate net salary
        net_salary = basic_salary + overtime - deductions

        # Create Payroll object
        payroll = Payroll(
            employee_id=emp_id,
            pay_period_start_date=start_date,
            pay_period_end_date=end_date,
            basic_salary=basic_salary,
            overtime_pay=overtime,
```

```
            deductions=deductions,
            net_salary=net_salary
        )

        # Pass Payroll object to service
        self.payroll_service.generate_payroll(payroll)
        print("Payroll generated.")

    def calculate_tax(self):
        emp_id = input("Enter Employee ID: ")
        year = input("Enter Tax Year (e.g., 2024): ")
        self.tax_service.calculate_tax(emp_id, year)
        print("Tax calculated.")

    def view_payrolls_for_employee(self):
        emp_id = input("Enter Employee ID: ")
        payrolls = self.payroll_service.get_payrolls_for_employee(emp_id)

        if not payrolls:
            print("No payroll records found.")
            return

        for p in payrolls:
            print(f"""
    Payroll ID: {p.get_payroll_id()}
    Employee ID: {p.get_employee_id()}
    Pay Period: {p.get_pay_period_start_date()} to
{p.get_pay_period_end_date()}
    Basic Salary: {p.get_basic_salary()}
    Overtime Pay: {p.get_overtime_pay()}
    Deductions: {p.get_deductions()}
    Net Salary: {p.get_net_salary()}
    ----------------------------""")

    def view_taxes_for_employee(self):
        emp_id = input("Enter Employee ID: ")
        taxes = self.tax_service.get_taxes_for_employee(emp_id)

        if not taxes:
            print("No tax records found.")
            return

        for tax in taxes:
            print(f"""
```

```
    Tax ID: {tax.get_tax_id()}
    Employee ID: {tax.get_employee_id()}
    Tax Year: {tax.get_tax_year()}
    Taxable Income: ₹{tax.get_taxable_income():.2f}
    Tax Amount: ₹{tax.get_tax_amount():.2f}
    ----------------------------""")

# Entry point
if __name__ == "__main__":
    MainModule().menu()
```

6.Test Module (test/):

The test module contains all the unit test cases written to verify the functionality of the system.
It ensures that each component works as expected and helps in identifying bugs early.

test_payxpert.py

The test_payxpert.py file includes test cases for important features like:

- Employee creation and validation
- Salary and tax calculations
- Error handling for invalid data
- Database operation checks

test_payxpert.py

```
import pytest
from decimal import Decimal
from entity.Employee import Employee
from entity.Payroll import Payroll
from entity.Tax import Tax
from dao.payroll_service import PayrollService
from dao.tax_service import TaxService
from util.database_context import DatabaseContext
from util.validation_service import ValidationService
from exception.custom_exceptions import InvalidInputException
```

```
#  Establish database connection for services
conn = DatabaseContext.get_connection()

# ---------- Test Case 1: Calculate Gross Salary ----------
def test_calculate_gross_salary_for_employee():
    basic = 60000.00
    overtime = 5000.00
    gross = basic + overtime
    assert gross == 65000.00

# ---------- Test Case 2: Calculate Net Salary ----------
def test_calculate_net_salary_after_deductions():
    basic = 60000.00
    overtime = 5000.00
    deductions = 2000.00
    net = basic + overtime - deductions
    assert net == 63000.00

# ---------- Test Case 3: Tax Calculation for High Income ----------
def test_verify_tax_calculation_for_high_income_employee():
    tax_service = TaxService(conn)
    high_income_tax = Tax(tax_id=1, employee_id=1, tax_year=2024,
taxable_income=Decimal('1200000.00'), tax_amount=Decimal('0.00'))
    high_income_tax.set_tax_amount(high_income_tax.get_taxable_inco
me() * Decimal('0.10'))
    assert high_income_tax.get_tax_amount() == Decimal('120000.00')

# ---------- Test Case 4: Process Payroll for Multiple Employees ----------
def test_process_payroll_for_multiple_employees():
    payroll_service = PayrollService(conn)
    employee_ids = [1, 2, 3]  # Use real existing IDs

    count = 0
    for i in employee_ids:
        payroll = Payroll(payroll_id=None, employee_id=i,
basic_salary=40000, overtime_pay=2000,
                          deductions=1000, net_salary=41000)
        payroll_service.generate_payroll(payroll)
        count += 1

    assert count == 3


# ---------- Test Case 5: Error Handling for Invalid Input ----------
```

```
def test_verify_error_handling_for_invalid_email():
    invalid_email = "john[at]email.com"
    with pytest.raises(Exception):
        if not ValidationService.validate_email(invalid_email):
            raise InvalidInputException("Invalid Email Format")
```

7.OUTPUT INTERFACE AND OUTPUTS

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

========== EMPLOYEE MANAGEMENT SYSTEM ==========
1. Add Employee
2. View All Employees
3. Get Employee by ID
4. Update Employee
5. Delete Employee
6. Generate Payroll
7. View Payrolls for an Employee
8. Calculate Tax
9. View Taxes for an Employee
10. Exit
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter your choice: 1

Enter Employee Details:
Employee ID: 15
First Name: prasanth
Last Name: A
Date of Birth (YYYY-MM-DD): 2003-12-12
Gender: male
Email: pom@gmail.com
Phone Number: 7401476613
Address: kk nagar
Position: web developer
Joining Date (YYYY-MM-DD): 2025-12-11
Termination Date (YYYY-MM-DD) or leave blank: 2026-12-12
Employee added successfully.
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter your choice: 2
ID: 1 | Name: John Doe | Email: john.doe@email.com
ID: 2 | Name: asus king | Email: asus@gmail.com
ID: 3 | Name: Mike Brown | Email: mike.brown@email.com
ID: 4 | Name: Sara Lee | Email: sara.lee@email.com
ID: 5 | Name: Tom Wilson | Email: tom.wilson@email.com
ID: 7 | Name: Robert King | Email: robert.king@email.com
ID: 8 | Name: Lucy Turner | Email: lucy.turner@email.com
ID: 9 | Name: Steve Jobs | Email: steve.jobs@email.com
ID: 10 | Name: Nina Patel | Email: nina.patel@email.com
ID: 15 | Name: prasanth A | Email: pom@gmail.com
ID: 21 | Name: prasanth A | Email: avc@gmail.com
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter your choice: 3
Enter Employee ID: 21

            Employee Details:
            ID: 21
            First Name: prasanth
            Last Name: A
            Date of Birth: 2002-12-12
            Gender: male
            Email: avc@gmail.com
            Phone Number: 1234567653
            Address: kk nagar
            Position: software
            Joining Date: 2021-12-12
            Termination Date: 2027-12-12
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter your choice: 4
Enter Employee ID to update: 21
Leave field blank to keep current value.
First Name: Prabu
Last Name: sundaram
Email: mnc@gmail.com
Phone Number: 0011001002
Employee updated.
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter your choice: 5
Enter Employee ID to delete: 4
Employee deleted.
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter your choice: 6
Enter Employee ID: 5
Enter Pay Period Start Date (YYYY-MM-DD): 2022-11-11
Enter Pay Period End Date (YYYY-MM-DD): 2025-11-11
Enter Basic Salary: 50000
Enter Overtime Pay: 2456
Enter Deductions: 3000
Payroll generated.
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter your choice: 7
Enter Employee ID: 5

    Payroll ID: 5
    Employee ID: 5
    Pay Period: 2024-03-01 to 2024-03-31
    Basic Salary: 65000.00
    Overtime Pay: 4000.00
    Deductions: 2200.00
    Net Salary: 66800.00
    ------------------------------

    Payroll ID: 15
    Employee ID: 5
    Pay Period: 2022-11-11 to 2025-11-11
    Basic Salary: 50000.00
    Overtime Pay: 2456.00
    Deductions: 3000.00
    Deductions: 3000.00
    Net Salary: 49456.00
    ------------------------------
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter your choice: 8
Enter Employee ID: 5
Enter Tax Year (e.g., 2024): 2025
Tax calculated.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Enter your choice: 9
Enter Employee ID: 5

    Tax ID: 5
    Employee ID: 5
    Tax Year: 2023
    Taxable Income: ₹780000.00
    Tax Amount: ₹78000.00
    ------------------------------

    Tax ID: 11
    Employee ID: 5
    Tax Year: 2024
    Taxable Income: ₹116256.00
    Tax Amount: ₹11625.60
    ------------------------------

    Tax ID: 12
    Employee ID: 5
    Tax Year: 2025
    Taxable Income: ₹116256.00
    Tax Amount: ₹11625.60
    ------------------------------
```